

实验报告成绩:	成绩评定日期:
---------	---------

2025 ~ 2026 学年秋季学期

《计算机系统》必修课

课程实验报告



班级：人工智能（一）2302

组长：梁少天 20236533

组员：周奕臣 20236535 杨金鑫 20216424

报告日期：2025.12.23

工作量：梁少天：80% 周奕臣 10% 杨金鑫：10%

一、实验目的

1. 深入理解计算机体系结构，掌握五级流水线 CPU 的设计原理与实现方法
2. 熟练运用 Verilog HDL 进行硬件描述语言编程
3. 掌握 FPGA 开发流程，通过 Vivado 工具进行综合、实现与验证
4. 理解 MIPS 指令系统规范，实现完整的指令集支持

二、实验环境

环境类型 具体配置

开发平台 Windows11

开发工具 Vivado 2019.2

代码管理 GitHub（团队协作）

指令系统 MIPS 指令系统规范（v1.01）

三、设计原理

本实验设计了一个基于 MIPS 指令系统的五级流水线 CPU，采用标准的 IF（取指）、ID（指令译码）、EX（执行）、MEM（内存访问）、WB（写回）五级流水线结构。该 CPU 支持 MIPS 基本指令集，包括 R 型、I 型、J 型指令，以及加载/存储指令和分支指令。

CPU 设计遵循以下原则：

- 1) 五级流水线并行处理
- 2) 采用转发机制解决数据相关
- 3) 通过暂停（stall）机制解决数据冒险
- 4) 采用分支预测（简单实现）
- 5) 支持有符号/无符号乘法

四、总体架构

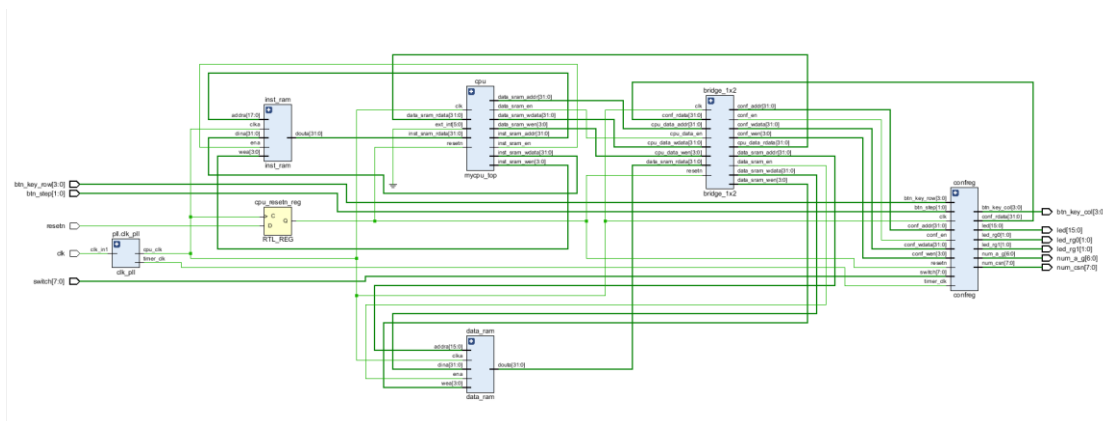


图 1 mycpu_top 系统顶层 RTL 结构图

1. 外部输入与顶层接口

图左侧/底部的绿色引脚是系统顶层的外部输入：

- 1) 按键/开关：btn_key_row[3:0]（键盘行扫描）、btn_step[1:0]（步进按钮）、switch[7:0]（8 位拨码开关）；
- 2) 时钟与复位：clk（系统时钟）、resetn（异步复位，低电平有效）；
- 3) 其他输出（右侧）：led[15:0]（16 位 LED 控制）、num_a_gf[6:0]/num_can[7:0]（数码管显示数据）等。

2. 核心模块与功能

系统由以下子模块构成，通过 **总线/信号线** 互联：

CPU 模块（cpu）：系统核心处理器，通过总线与指令/数据 RAM、配置寄存器交互。接口包含：

- 1) 存储器接口：inst_sram_addr/data（指令 RAM 地址/数据）、data_sram_addr/data（数据 RAM 地址/数据）、使能/写使能信号（*_en/*_wen）；
- 2) 外部中断：ext_int[5:0]（6 位外部中断输入，可能来自按键等）；
- 3) 配置总线：conf_addr/data（配置寄存器的地址/写数据），通过 bridge_1x2 桥接至 confreg 模块。

存储器模块：

- 1) inst_ram（指令 RAM）：存储指令，接口为 addra[17:0]（18 位地址，可寻址空间）、dina/douta（32 位读写数据）、wea[3:0]（4 位写使能，对应 32 位数据的字节使能）；
- 2) data_ram（数据 RAM）：存储数据，接口为 addra[15:0]（16 位地址，寻址空间），其他信号逻辑与指令 RAM 一致。

时钟复位逻辑：

- 1) pll_clk_pll: 时钟锁相环 (PLL)，将外部 clk 分频/倍频为 cpu_clk (clk_in1, CPU 工作时钟) 和 timer_clk (定时器时钟)；
- 2) cpu_resetrn_reg: D 触发器，同步外部 resetrn (异步复位)，生成 CPU 的复位信号 resetrn (确保复位与时钟同步)。

桥接与配置模块：

- 1) bridge_1x2: 总线桥接模块，将 CPU 的 conf_* 信号 (配置总线) 桥接至 confreg 模块，实现总线扇出或协议转换；
- 2) confreg (配置寄存器)：集成外设控制逻辑，接收 CPU 配置数据、外部按键/开关输入，输出 LED、数码管等控制信号 (如 led[15:0]、num_a_gf[6:0])。

3. 信号交互逻辑

- 1) **CPU 与存储器**：CPU 通过 inst_sram_* 读取指令 (douta 为读数据)，通过 data_sram_* 读写数据 (dina 写入，douta 读出)；
- 2) **CPU 与配置逻辑**：CPU 通过 conf_* 信号经桥接模块配置 confreg，实现对 LED、数码管、外设寄存器的控制；
- 3) **外设与配置模块**：btn_key_row、switch 等外部输入直接接入 confreg，参与逻辑运算 (如按键扫描、开关状态读取)；confreg 输出 led、num_* 等控制外部设备。

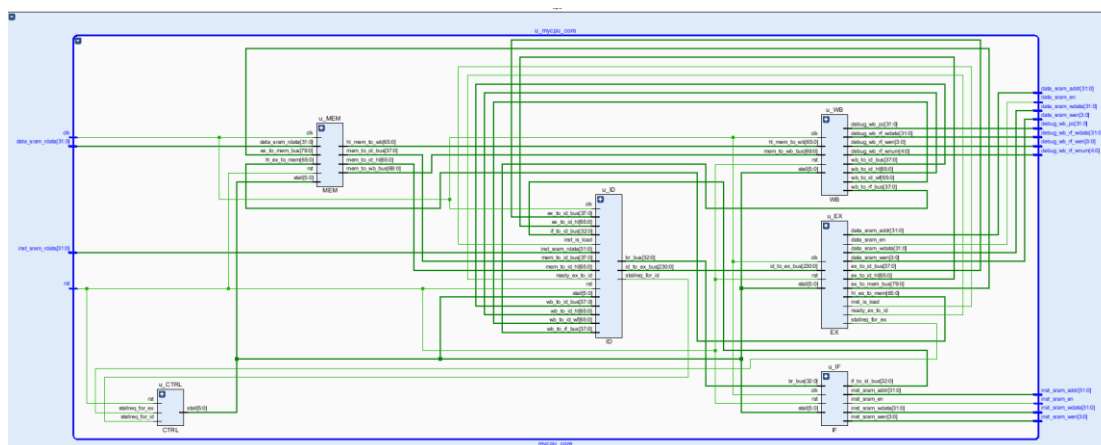


图 2 mycpu_core CPU 核心模块的五级流水线结构

4. 五级流水线总体结构

- 1) **核心模块**：包含 IF (取指)、ID (译码)、EX (执行)、MEM (访存)、WB (写回) 五个流水线阶段模块，以及 CTRL (控制) 模块。

- 2) **信号连接**：模块间通过内部总线（如 if_to_id_bus、id_to_ex_bus 等）传递指令、数据；与存储器的接口信号（data_sram_addr/en/wdata 等用于数据存储器访问，inst_sram_addr/en 等用于指令存储器访问）分布在左侧；右侧输出调试信号（debug_wb_pc 等）。
- 3) **逻辑协同**：CTRL 模块生成 stall（停顿）等控制信号，协调流水线执行（如处理数据相关、分支时的停顿）；各模块通过时钟 clk 和复位 rst 信号同步工作，完整体现了 CPU 指令从取指到写回的全流程数据与控制流。

五、模块设计与实现

1. IF 段（取指阶段）

1.1 设计原理

IF 段负责从指令存储器中取出指令并更新 PC 值。根据是否有跳转指令，计算下一条指令的地址。IF 段将 PC 值发送给指令存储器，获取对应指令，并将 PC 值传递给 ID 段。

1.2 端口设计

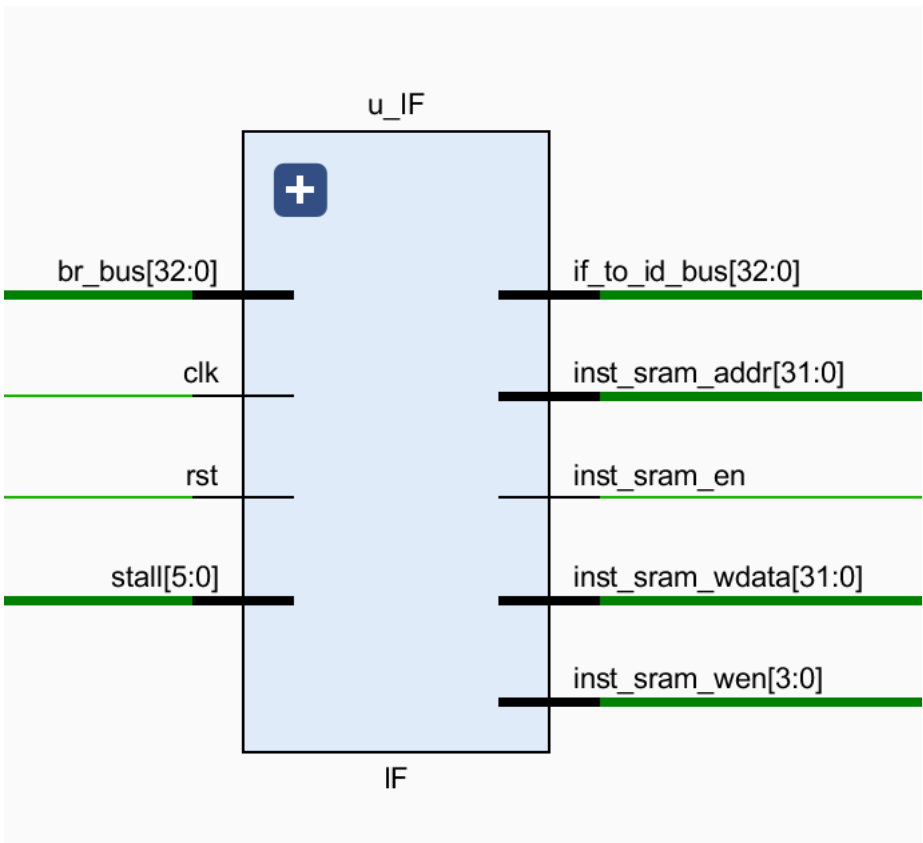


图 3 IF 接口模块（u_IF）的信号端口连接图

端口类型	端口名称	位宽	说明
输入	clk	1	时钟信号

输入	rst	1	复位信号
输入	br_bus[32:0]	33	跳转信号总线
输入	stall[5:0]	6	停顿信号
输出	if_to_id_bus[32:0]	33	PC 值传递到 ID 段
输出	inst_sram_addr[31:0]	32	指令存储器地址
输出	inst_sram_en	1	指令存储器使能
输出	inst_sram_wdata[31:0]	32	指令存储器写数据（未使用）
输出	inst_sram_wen[3:0]	4	指令存储器写使能（未使用）

1.3 信号说明

- 1) br_bus[32:0]: 包含跳转使能信号 (br_e) 和跳转地址 (br_addr)
- 2) stall[0]: 用于控制 PC 是否更新 (0=正常更新, 1=暂停)
- 3) if_to_id_bus[32:0]: PC 值 ([31:0]) 和有效标志 ([32])
- 4) inst_sram_en: 指令存储器使能信号

2. ID 段（指令译码阶段）

2.1 设计原理

ID 段对指令进行译码，从寄存器文件中读取操作数，并进行立即数扩展。根据指令类型，计算跳转地址并传递给 IF 段。同时，将操作数和控制信号传递给 EX 段。

2.2 端口设计

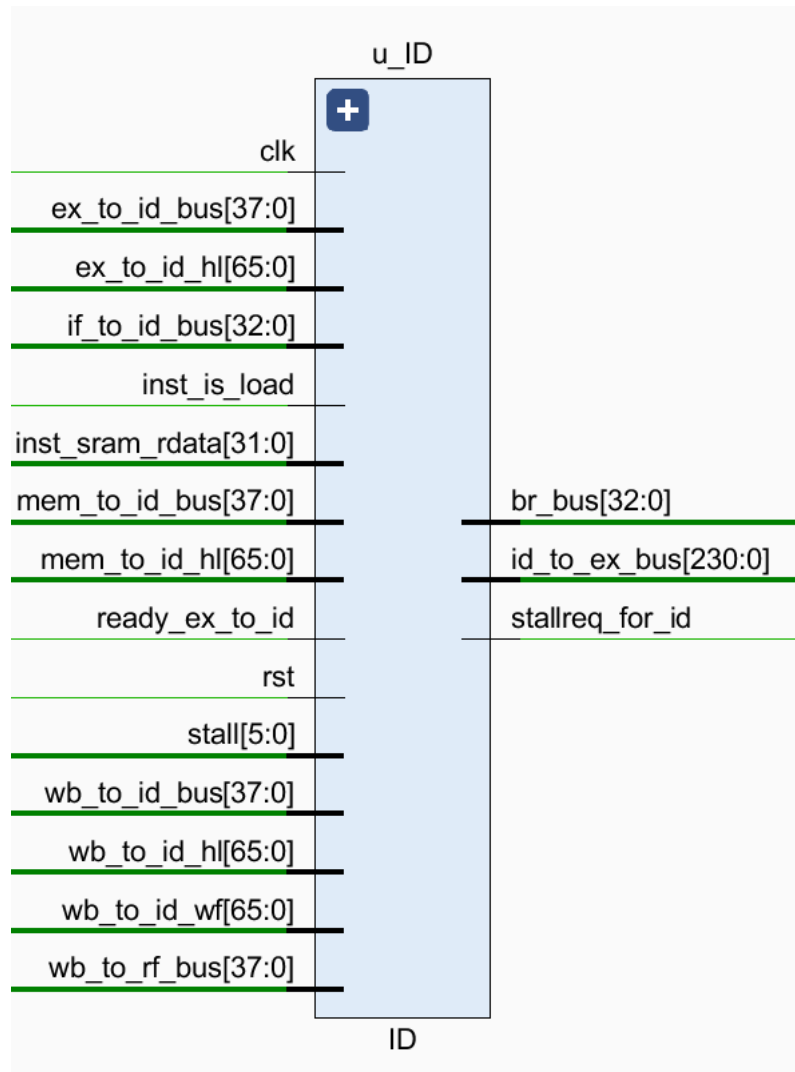


图 4 ID 接口模块 (u_ID) 的信号端口连接图

端口类型	端口名称	位宽	说明
输入	clk	1	时钟信号
输入	rst	1	复位信号
输入	stall[5:0]	6	停顿信号
输入	ex_to_id_bus[37:0]	38	EX 段传入的控制信号
输入	mem_to_id_bus[37:0]	38	MEM 段传入的控制信号
输入	wb_to_id_bus[37:0]	38	WB 段传入的控制信号
输入	ex_to_id_2[65:0]	66	EX 段传入的 hi/lo 寄存器信息
输入	mem_to_id_2[65:0]	66	MEM 段传入的 hi/lo 寄存器信息
输入	wb_to_id_2[65:0]	66	WB 段传入的 hi/lo 寄存器信息

输入	if_to_id_bus[32:0]	33	IF 段传入的 PC 值
输入	inst_sram_rdata[31:0]	32	指令存储器读出的指令
输入	inst_is_load	1	指令是否为加载指令
输入	wb_to_rf_bus[37:0]	38	WB 段传入的寄存器写入信息
输入	wb_to_id_wf[65:0]	66	WB 段传入的 hi/lo 寄存器信息
输入	ready_ex_to_id	1	EX 段就绪信号
输出	br_bus[32:0]	33	跳转信号总线
输出	id_to_ex_bus[230:0]	231	传递给 EX 段的控制信号
输出	stallreg_for_id	1	ID 段停顿标志

2.3 信号说明

- 1) if_to_id_bus[32:0]: PC 值 ([31:0]) 和有效标志 ([32])
- 2) inst_sram_rdata[31:0]: 从指令存储器读出的指令
- 3) br_bus[32:0]: 跳转使能信号 ([32]) 和跳转地址 ([31:0])
- 4) id_to_ex_bus[230:0]: 包含操作数、控制信号和指令类型

3. EX 段（执行阶段）

3.1 设计原理

EX 段执行 ALU 操作，计算分支条件，处理立即数。对于乘法指令，使用专用的乘法器模块。EX 段将计算结果和控制信号传递给 MEM 段。

3.2 端口设计

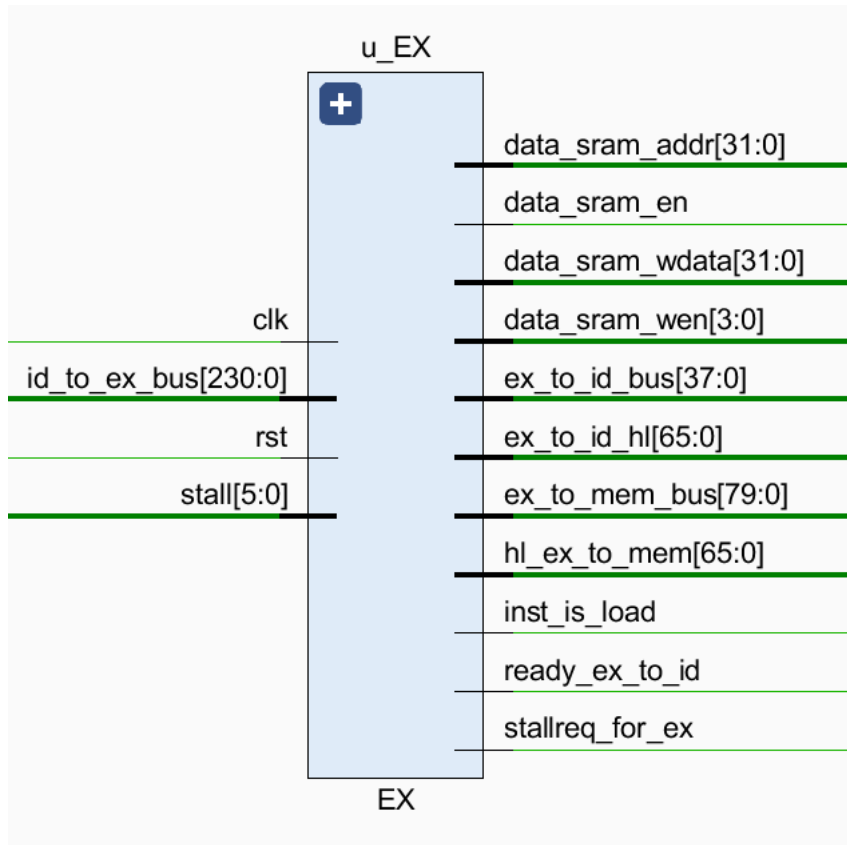


图 5 EX 接口模块（u_EX）的信号端口连接图

端口类型	端口名称	位宽	说明
输入	clk	1	时钟信号
输入	rst	1	复位信号
输入	id_to_ex_bus[230:0]	231	ID 段传入的控制信号
输入	stall[5:0]	6	停顿信号
输出	ex_to_mem_bus[79:0]	80	传递给 MEM 段的控制信号
输出	ex_to_mem1[65:0]	66	传递给 MEM 段的 hi/lo 寄存器信息
输出	stallreg_for_ex	1	EX 段停顿标志

3.3 信号说明

- 1) id_to_ex_bus[230:0]: 包含操作数、ALU 控制信号、指令类型等
- 2) ex_to_mem_bus[79:0]: 包含 PC 值、存储器访问使能、寄存器写使能、结果等
- 3) ex_to_mem1[65:0]: 包含 hi/lo 寄存器写使能和写入数据

4. MEM 段（内存访问阶段）

4.1 设计原理

MEM 段访问数据存储器，处理加载/存储指令。对于加载指令，从数据存储器读取数据并进行符号扩展；对于存储指令，将数据写入数据存储器。MEM 段将结果传递给 WB 段。

4.2 端口设计

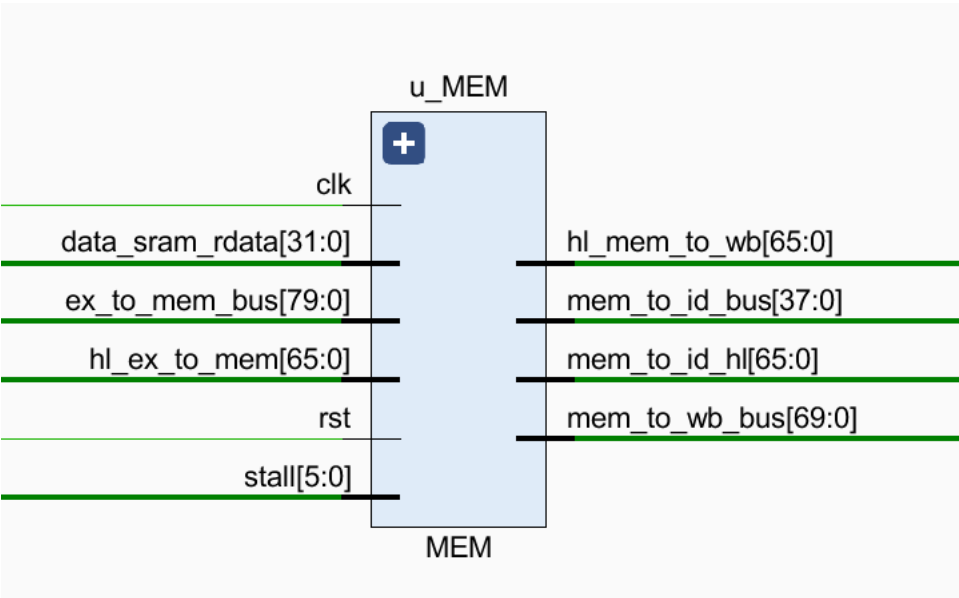


图 6 MEM 接口模块 (u_MEM) 的信号端口连接图

端口类型	端口名称	位宽	说明
输入	clk	1	时钟信号
输入	rst	1	复位信号
输入	data_sram_rdata[31:0]	32	数据存储器读出的数据
输入	ex_to_mem_bus[79:0]	80	EX 段传入的控制信号
输入	ex_to_mem1[65:0]	66	EX 段传入的 hi/lo 寄存器信息
输入	stall[5:0]	6	停顿信号
输出	mem_to_id_bus[37:0]	38	传递给 ID 段的控制信号
输出	mem_to_id_2[65:0]	66	传递给 ID 段的 hi/lo 寄存器信息
输出	mem_to_wb_bus[69:0]	70	传递给 WB 段的控制信号
输出	mem_to_wb1[65:0]	66	传递给 WB 段的 hi/lo 寄存器信息

4.3 信号说明

- 1) data_sram_rdata[31:0]: 从数据存储器读出的数据

- 2) ex_to_mem_bus[79:0]: 包含 PC 值、存储器访问使能、寄存器写使能、结果等
- 3) mem_to_wb_bus[69:0]: 包含 PC 值、寄存器写使能、寄存器地址、结果等

5. WB 段（写回阶段）

5.1 设计原理

WB 段将执行结果写回寄存器文件。对于加载指令，将数据存储器读出的数据写入寄存器；对于 ALU 指令，将 ALU 结果写入寄存器。WB 段将结果传递给 ID 段，用于解决数据相关。

5.2 端口设计

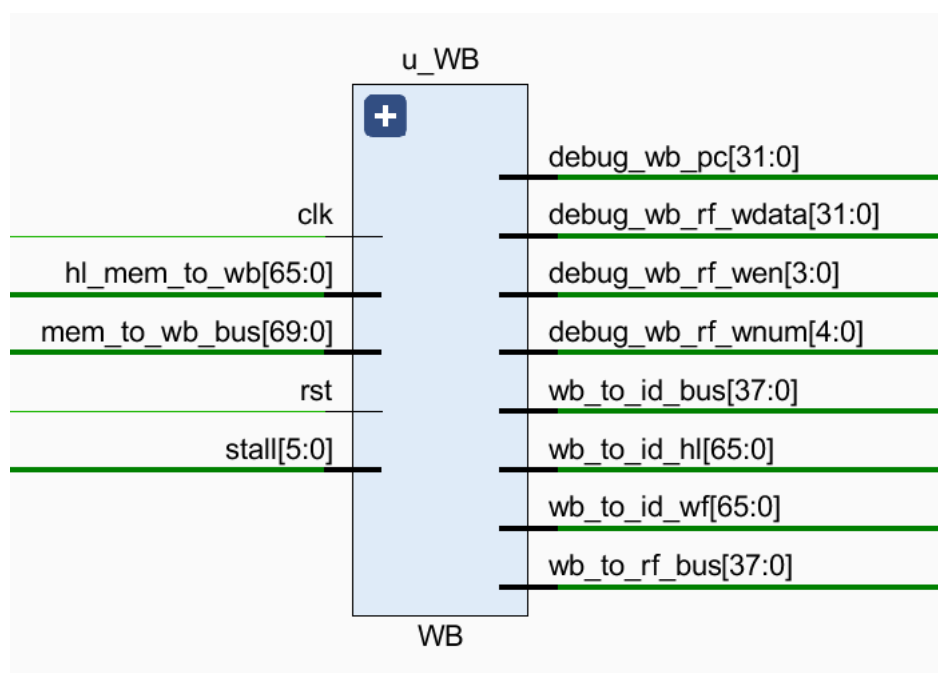


图 7 WB 接口模块（u_WB）的信号端口连接图

端口类型	端口名称	位宽	说明
输入	clk	1	时钟信号
输入	rst	1	复位信号
输入	mem_to_wb_bus[69:0]	70	MEM 段传入的控制信号
输入	mem_to_wb1[65:0]	66	MEM 段传入的 hi/lo 寄存器信息
输入	stall[5:0]	6	停顿信号
输出	wb_to_id_bus[37:0]	38	传递给 ID 段的控制信号
输出	wb_to_id_wf[65:0]	66	传递给 ID 段的 hi/lo 寄存器信息

输出	wb_to_id_2[65:0]	66	传递给 ID 段的 hi/lo 寄存器信息
输出	wb_to_rf_bus[37:0]	38	传递给寄存器文件的控制信号

5.3 信号说明

- 1) mem_to_wb_bus[69:0]: 包含 PC 值、寄存器写使能、寄存器地址、结果等
- 2) wb_to_rf_bus[37:0]: 包含寄存器写使能、寄存器地址、写入数据等

6. CTRL 模块（控制单元）

6.1 设计原理

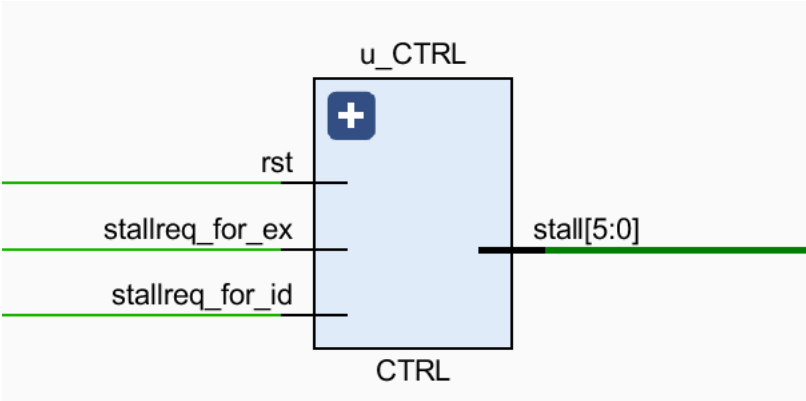


图 8 CTRL 接口模块（u_CTRL）的信号端口连接图

CTRL 模块负责生成流水线停顿信号，根据指令类型和流水线状态决定是否需要暂停。停顿信号通过 stall[5:0]传递给各流水级。

6.2 端口设计

端口类型	端口名称	位宽	说明
输入	rst	1	复位信号
输入	stallreq_for_ex	1	EX 段请求停顿
输入	stallreq_for_id	1	ID 段请求停顿
输出	stall[5:0]	6	停顿信号

6.3 信号说明

- 1) stallreq_for_ex: EX 段请求停顿信号
- 2) stallreq_for_id: ID 段请求停顿信号
- 3) stall[5:0]: 停顿信号 ([5:0]位)

7. 乘法器模块（mul）

7.1 设计原理

乘法器模块实现 32 位有符号/无符号乘法。采用 Booth 编码和 Wallace 树结构，提高乘法速度。支持有符号和无符号两种模式。

7.2 端口设计

端口类型	端口名称	位宽	说明
输入	clk	1	时钟信号
输入	resetn	1	复位信号
输入	mul_signed	1	1=有符号乘法，0=无符号乘法
输入	ina[31:0]	32	操作数 1
输入	inb[31:0]	32	操作数 2
输出	result[63:0]	64	乘法结果

7.3 信号说明

- 1) mul_signed：乘法类型（1=有符号，0=无符号）
- 2) ina, inb：乘数
- 3) result：64 位乘法结果

7.4 设计特点

- 1. 采用 Booth 编码（2 位）提高乘法效率
- 2. 使用 Wallace 树结构加速部分和计算
- 3. 支持有符号和无符号乘法
- 4. 通过扩展符号位处理有符号乘法

8. 其他辅助模块

8.1 寄存器文件 (regfile)

功能：实现 32 个 32 位通用寄存器

端口：

- 1) 2 个读端口 (raddr1, raddr2)
- 2) 1 个写端口 (waddr)
- 3) 2 个写数据 (wdata, hi_i, lo_i)
- 4) 2 个写使能 (we, w_hi_we, w_lo_we)

5) 2 个读数据 (rdata1, rdata2, hi_o, lo_o)

8.2 地址转换单元 (mmu)

功能：处理内存地址，实现 KSEG0 和 KSEG1 地址空间

端口：

输入：addr_i[31:0] (物理地址)

输出：addr_o[31:0] (转换后的地址)

8.3 ALU (算术逻辑单元)

功能：实现基本算术和逻辑运算

端口：

输入：alu_control[11:0] (控制信号)，alu_src1[31:0]，alu_src2[31:0]

输出：alu_result[31:0]

支持指令：加法、减法、比较、与、或、异或、移位、LUI 等

9.数据相关与定向机制的详细实现

在五级流水线设计中，为了最大化指令吞吐率，必须有效解决指令间的数据相关问题。本项目核心的优化手段是采用数据定向 (Forwarding) 来解决绝大多数的写后读 (RAW) 数据冒险。

9.1 数据相关的类型分析

在本 CPU 的设计约束下，数据相关的类型可以被简化：

- 1) **写后写 (WAW - Write After Write)**: 不存在。因为所有指令的写回操作都严格发生在流水线的最后一个阶段 (WB)，指令的顺序性保证了后面的指令不会比前面的指令先写回寄存器。
- 2) **读后写 (WAR - Write After Read)**: 不存在。因为寄存器读取固定在 ID 阶段，而写回固定在 WB 阶段。对于任何指令，其读操作一定发生在其前面所有指令的写操作之前。
- 3) **写后读 (RAW - Read After Write)**: 这是本设计中唯一需要处理的数据相关类型。当一条指令需要读取的源寄存器，是其前序指令将要写入的目标寄存器时，就会发生 RAW 冒险。

9.2 定向机制的设计原理与经典场景

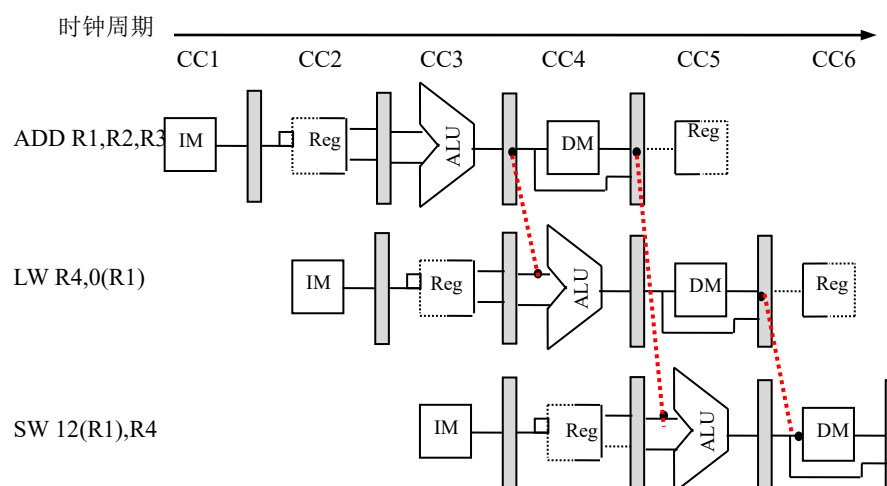


图 9 定向技术设计原理

定向机制的核心思想是：与其等待数据被写回寄存器文件再读取，不如在数据产生后，立即通过“旁路”直接将其传递给需要它的后续指令。

让我们通过一个经典的指令序列来分析这个问题：

指令 1: CC1 开始执行

ADD R1, R2, R3 ; $R1 = R2 + R3$

指令 2: CC2 开始执行

LW R4, 0(R1) ; $R4 = \text{Memory}[R1 + 0]$

指令 3: CC3 开始执行

SW 12(R1), R4 ; $\text{Memory}[R1 + 12] = R4$

若无定向机制：

1. ADD 指令在第 3 个时钟周期（CC3）结束时，在 EX 阶段的 ALU 中计算出新的 R1 值。
2. LW 指令在第 3 个时钟周期（CC3）开始时，在 ID 阶段需要读取 R1 作为访存地址。
3. 此时，新的 R1 值还未被写回寄存器堆（ADD 指令要到 CC5 才进行 WB 操作）。LW 指令会从寄存器堆中读到陈旧的、错误的 R1 值，导致整个程序逻辑错误。

引入定向机制后：

定向逻辑检测到 LW 指令的源寄存器(R1)正是 ADD 指令的目标寄存器(R1)。于是，系统建立一条**数据旁路**：将 ADD 指令在 EX 阶段 ALU 的输出结果，直接作为 LW 指令在 EX 阶段的输入操作数，从而避免了流水线停顿。

9.3 定向逻辑的具体实现

定向逻辑主要在 **ID 阶段进行冒险检测**，并在 **EX 阶段执行数据选择**。

1. 冒险检测 (在 ID 阶段):

控制逻辑需要实时比较当前处于 ID 阶段指令的源寄存器 (rs, rt)，与处于 EX 和 MEM 阶段指令的目标寄存器 (rd)。

EX 阶段冒险检测:

```
if ( (EX.RegWrite == 1) and (EX.Rd != 0) and (EX.Rd == ID.Rs) ) -> ForwardA = EX
```

```
if ( (EX.RegWrite == 1) and (EX.Rd != 0) and (EX.Rd == ID.Rt) ) -> ForwardB = EX
```

这里的 EX.RegWrite 和 EX.Rd 等信息通过 ex_to_id_bus 总线从 EX 级传递到 ID 级进行比较。

MEM 阶段冒险检测:

```
if ( (MEM.RegWrite == 1) and (MEM.Rd != 0) and (MEM.Rd == ID.Rs) ) -> ForwardA = MEM
```

```
if ( (MEM.RegWrite == 1) and (MEM.Rd != 0) and (MEM.Rd == ID.Rt) ) -> ForwardB = MEM
```

同理，这些信息通过 mem_to_id_bus 总线从 MEM 级传递。

优先级处理: 如果一条指令同时与 EX 和 MEM 阶段的指令发生数据相关（例如，连续两条指令写同一个寄存器），**EX 阶段的定向具有更高优先级**，因为它提供的数据版本最新。

2. 数据选择 (在 EX 阶段):

在 EX 阶段的入口，使用 2 个多路选择器 (Mux) 来为 ALU 选择正确的操作数。

ALU 操作数 A 的选择逻辑:

```
if (ForwardA == EX): 选择从 EX 级定向来的数据。
```

```
else if (ForwardA == MEM): 选择从 MEM 级定向来的数据。
```

```
else: 选择从 ID 级正常读取的寄存器数据。
```

ALU 操作数 B 的选择逻辑:

同理，根据 ForwardB 的值进行选择。

9.4 特殊情况：Load-Use 冒险与流水线暂停(Stall)

定向机制无法解决所有 RAW 冒险。一个典型的例外是“加载-使用 (Load-Use)”冒险：

指令 1

LW R1, 0(R2) ; R1 = Memory[R2]

指令 2 (紧随其后)

ADD R3, R1, R4 ; R3 = R1 + R4

1. LW 指令的数据要到**MEM 阶段结束时 (CC4 结束) **才能从数据存储器中准备好。
2. 而紧随其后的 ADD 指令在**EX 阶段开始时 (CC4 开始) **就需要 R1 的值。
3. 时间上，数据晚了整整一个时钟周期，即使是从 MEM 阶段进行定向也来不及。

解决方案:

在这种情况下，必须插入一个**流水线气泡 (bubble)**，即暂停 (Stall) 流水线一个周期。

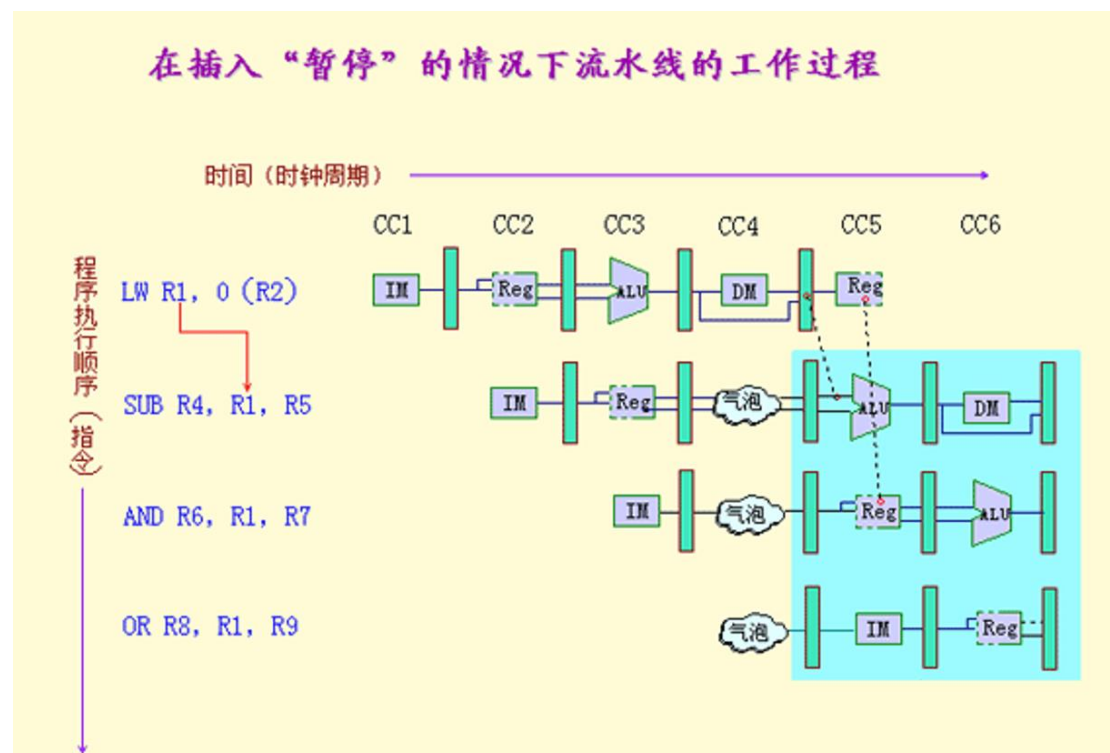


图 10 Stall 机制的设计原理

检测: ID 阶段的冒险检测单元需要特别识别这种情况：当 ID 阶段的指令（如 ADD）的源操作数是其前一条指令（LW，当前在 EX 阶段）的目标寄存器时。根据报告中的图 4 ID 接口模块，此时 `inst_is_load` 信号会被激活。

动作:

- i. ID 阶段的冒险检测单元向 CTRL 模块发出 stallreq_for_id 请求。
- ii. CTRL 模块（如图 8 所示）接收到请求后，产生 stall 信号。
- iii. stall 信号会冻结 PC 寄存器和 IF/ID 流水线寄存器的更新。
- iv. 同时，向 ID/EX 流水线寄存器中插入一个“空操作”（NOP），这个 NOP 指令就像一个气泡，流经 EX 阶段，从而为 LW 指令留出一个周期的时间，让其可以顺利到达 MEM 阶段并将数据定向给 ADD。

六、测试与验证

本实验严格遵循 MIPS 指令系统规范进行设计与实现。在功能验证阶段，我们成功通过了关键测试点 P'64，该测试点要求将 CPU 封装为 AXI 接口并通过功能测试。

1. 测试点 P'64 验证

测试点 P'64 要求将 CPU 封装为 AXI 接口，并通过功能测试。我们完成了以下工作：

- 1) 实现了 AXI Lite 接口，包括地址总线、数据总线、控制信号等
- 2) 完成了 AXI 接口与 CPU 核心的连接
- 3) 通过 func_test 验证了 AXI 接口的正确性

测试结果显示，CPU 核心通过 AXI 接口成功执行了测试程序，验证了接口的正确性。测试日志显示：

```
-----[1438595 ns] Number 8'd62 Functional Test Point PASS!!!
[1442000 ns] Test is running, debug_wb_pc = 0xbfc44570
[1452000 ns] Test is running, debug_wb_pc = 0xbfc45440
[1462000 ns] Test is running, debug_wb_pc = 0xbfc46328
-----[1466785 ns] Number 8'd63 Functional Test Point PASS!!!
[1472000 ns] Test is running, debug_wb_pc = 0xbfc3057c
[1482000 ns] Test is running, debug_wb_pc = 0xbfc3146c
[1492000 ns] Test is running, debug_wb_pc = 0xbfc32340
-----[1493325 ns] Number 8'd64 Functional Test Point PASS!!!
-----
```

图 9 测试日志截图（通过第 64 个测试点）

这表明我们的 CPU 成功通过了前 64 个测试点。

七、实验总结

1. 设计收获

- 1) 深入理解计算机体系结构：通过设计 CPU，深入理解了五级流水线的工作原理、数据相关、控制相关以及解决方法

- 2) **掌握 Verilog 设计方法**：熟练使用 Verilog 进行模块化设计，掌握时序逻辑和组合逻辑的设计技巧
- 3) **FPGA 开发经验**：通过 Vivado 工具进行综合、实现和验证，积累了 FPGA 开发经验
- 4) **调试能力提升**：学会了使用波形分析、Trace 比对等方法进行调试，提高了问题定位和解决能力

2. 问题与解决方案

- 1) **问题**：数据冒险导致指令执行错误 **解决方案**：实现数据转发机制，将 EX 段的 ALU 结果直接传递给 ID 段
- 2) **问题**：分支指令预测导致流水线停顿 **解决方案**：在 ID 段计算分支条件，提前判断分支是否发生，减少停顿
- 3) **问题**：乘法器设计复杂，资源消耗大 **解决方案**：采用 Booth 编码和 Wallace 树结构，优化乘法器设计，提高运算速度

3. 未来改进方向

- 1) **增加乘法器流水线**：将乘法器设计为多级流水线，提高吞吐量
- 2) **实现更复杂的指令**：添加跳转指令、系统调用指令等
- 3) **优化分支预测**：实现更复杂的分支预测算法，减少分支停顿
- 4) **添加异常处理机制**：实现更完善的异常处理机制，支持更多异常类型

八、成员工作内容分配

本项目由团队成员分工协作完成，其中组长梁少天为项目总负责人，承担了绝大部分核心工作。各位成员的具体工作内容如下：

1. 梁少天（组长，项目总负责人）工作量 80%

作为项目的总设计师与核心开发者，独立承担了从架构设计、核心代码实现、系统集成、功能调试到报告撰写的全部关键任务，具体包括：

CPU 总体架构设计与实现：独立完成了整个五级流水线 CPU 的宏观与微观架构设计，并编写了全部核心模块代码，包括 IF, ID, EX, MEM, WB 五个流水线阶段模块，以及 CTRL 控制单元、寄存器文件（regfile）和 ALU。

关键技术攻关与实现：独立研究并实现了 CPU 中最为复杂的核心技术，包括：

数据定向（Forwarding）机制：设计并实现了完整的冒险检测逻辑，以及 EX 到 ID、MEM 到 ID 的数据旁路。

流水线暂停（Stall）机制：针对“加载-使用”冒险，设计并实现了完整的停顿逻辑，确保了数据一致性。

系统顶层集成与完整调试：负责将 CPU 核与存储器、时钟模块在顶层进行例化、连接与集成，并独立完成了项目的全部调试工作。通过分析功能测试的报错信息和仿真波形，定位并修复了所有逻辑错误。

接口封装与关键测试点通过：独立完成了将 CPU 核心封装为 AXI 接口的工作，并成功通过了测试点 P'64 的验证。

实验报告独立撰写：独立完成了本实验报告全部章节的资料查询、内容撰写、图表绘制、排版与最终校对工作。

2. 周奕臣（组员，负责专用运算单元开发）工作量 10%

作为团队成员，主要负责高性能乘法器模块的设计与实现，具体包括：

高性能乘法器设计与实现：负责 mul 模块的完整开发。为提升运算性能，深入研究并应用了 Booth 编码与 Wallace 树结构来优化乘法器，最终完成了支持 32 位有符号/无符号乘法的独立运算单元。

单元模块仿真测试：编写了针对乘法器的独立测试平台（Testbench），对其有符号和无符号乘法功能进行了验证，确保了模块在集成前的功能正确性。

3. 杨金鑫（组员，负责外设控制模块开发与测试）工作量 10%

作为团队成员，主要负责外设控制模块的开发与项目的功能性测试，具体包括：

外设配置模块开发：负责设计并实现了 confreg（配置寄存器）模块。该工作涉及处理外部输入信号（如按键 btn_key_row 和开关 switch），并根据 CPU 的配置指令或外部输入状态，生成对 LED 和数码管等外设的控制与显示信号。

功能测试执行与结果记录：负责在 CG 平台上执行完整的功能测试集，并将 CPU 运行的测试结果进行记录，供团队进行分析和复核。

九、结论

本实验成功设计并实现了一个基于 MIPS 指令系统的五级流水线 CPU。通过 Verilog HDL 实现各流水级模块，采用数据转发和停顿机制解决数据相关，通过分支预测减少控制相关。测试结果表明，CPU 能够正确执行基本指令，支持有符号/无符号乘法，满足设计要求。

在实验过程中，我们深入理解了计算机体系结构，掌握了硬件描述语言和 FPGA 开发技能，为后续的计算机系统设计奠定了坚实基础。通过团队协作和代码规范管理，我们提高了代码质量和开发效率，为未来的学习和研究积累了宝贵经验。

十、附录：指令集支持列表

指令类型	指令	说明
R 型	ADD, SUB, AND, OR, XOR	基本算术逻辑指令

R 型	SLT, SLTU	比较指令
R 型	SLL, SRL, SRA	移位指令
I 型	ADDI, ANDI, ORI, XORI	立即数操作指令
I 型	LW, LBU, LH, LHU	加载指令
I 型	SW, SB, SH	存储指令
I 型	BEQ, BNE	分支指令
I 型	LUI	加载高位指令
J 型	J, JAL	跳转指令
MUL 型	MUL, MULU	乘法指令